

CS & IT ENGINEERING

DATA STRUCTURE

Binary Heap



JOIN TELEGRAM TO ENROLL

SHAIENDRA SIR

Binary Heap

- ✓ Binary Heap
- ✓ Heapify Procedure
- ✓ Build Binary Heap
- ✓ Operation on Binary Heap
- ✓ PYQ

Heapify Procedure:- To maintain Heap property

- max heap: $\text{Parent} \geq \text{left \& Right}$
- min heap: $\text{Parent} \leq \text{left \& Right}$



Heapify(A, i)

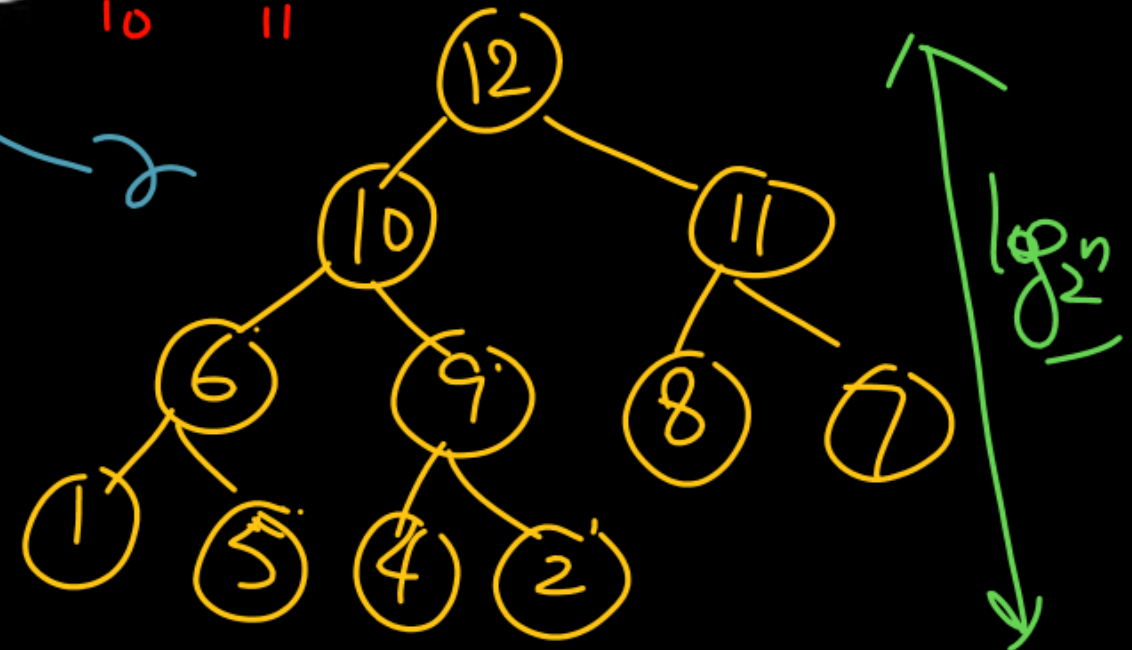
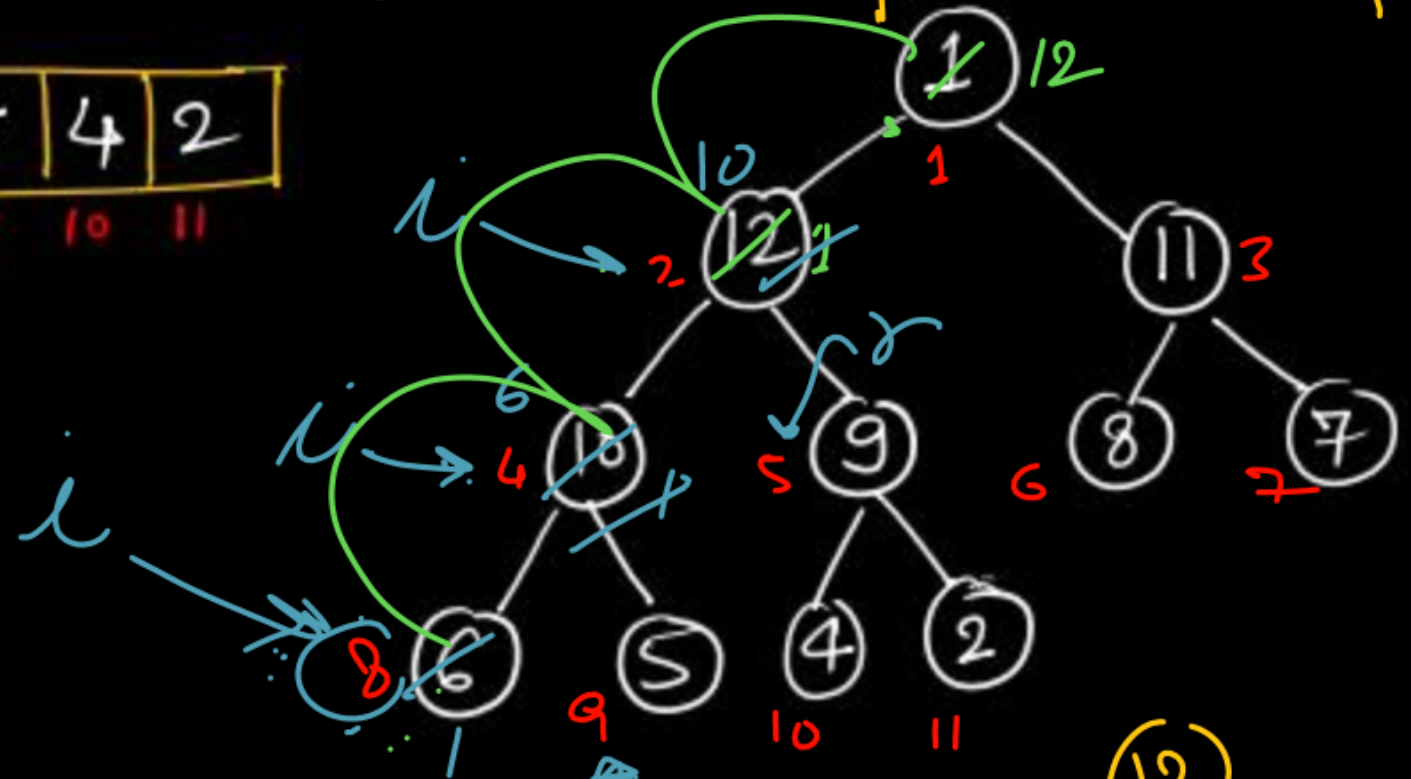
$l \leftarrow \text{left}(i) // 2i$

$r \leftarrow \text{Right}(i) // 2i + 1$

Heapify(A, largest)

$l = 16$

$r = 17$



Algo: Heapify(A, i)

1 - $l \leftarrow \text{Left}(i)$ // 2 // 4

2 - $r \leftarrow \text{Right}(i)$ // 3 // 5

3 - if $l \leq \text{heapsize}[A]$ and $A[l] > A[i]$

4 - Then $\text{largest} \leftarrow l$

5 - else $\text{largest} \leftarrow i$

6 - if $r \leq \text{heapsize}[A]$ and $A[r] > A[\text{largest}]$

7 - Then $\text{largest} \leftarrow r$

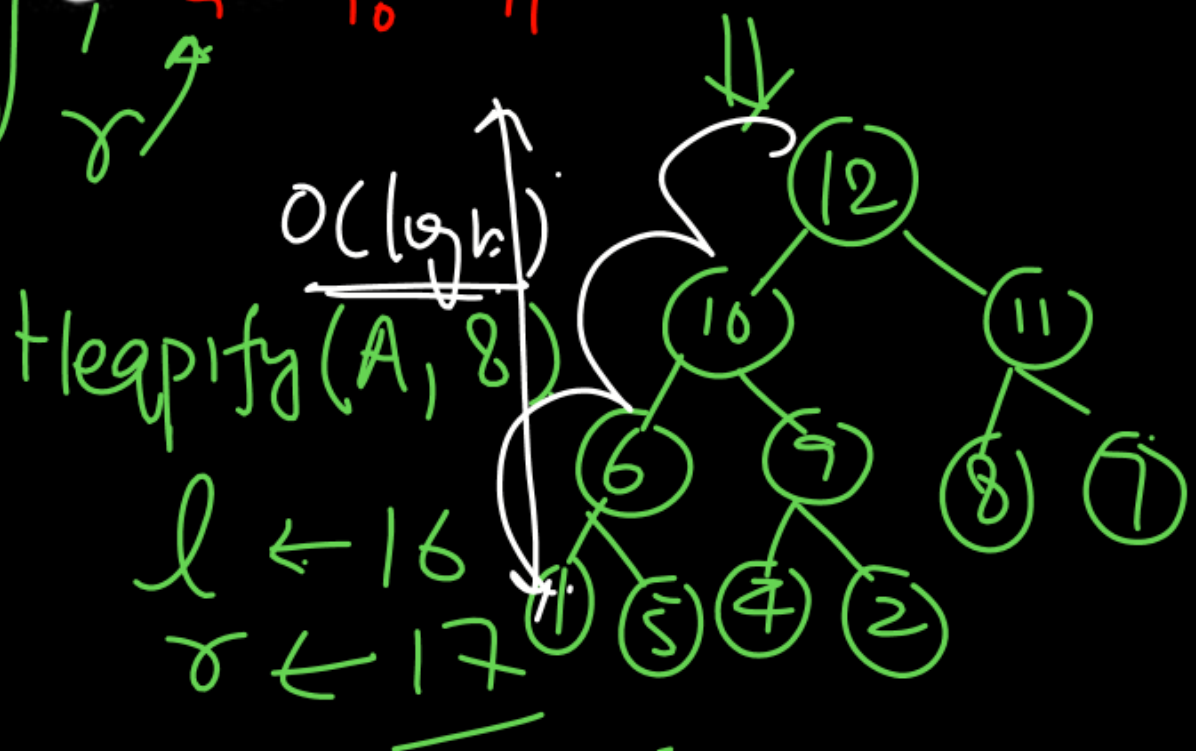
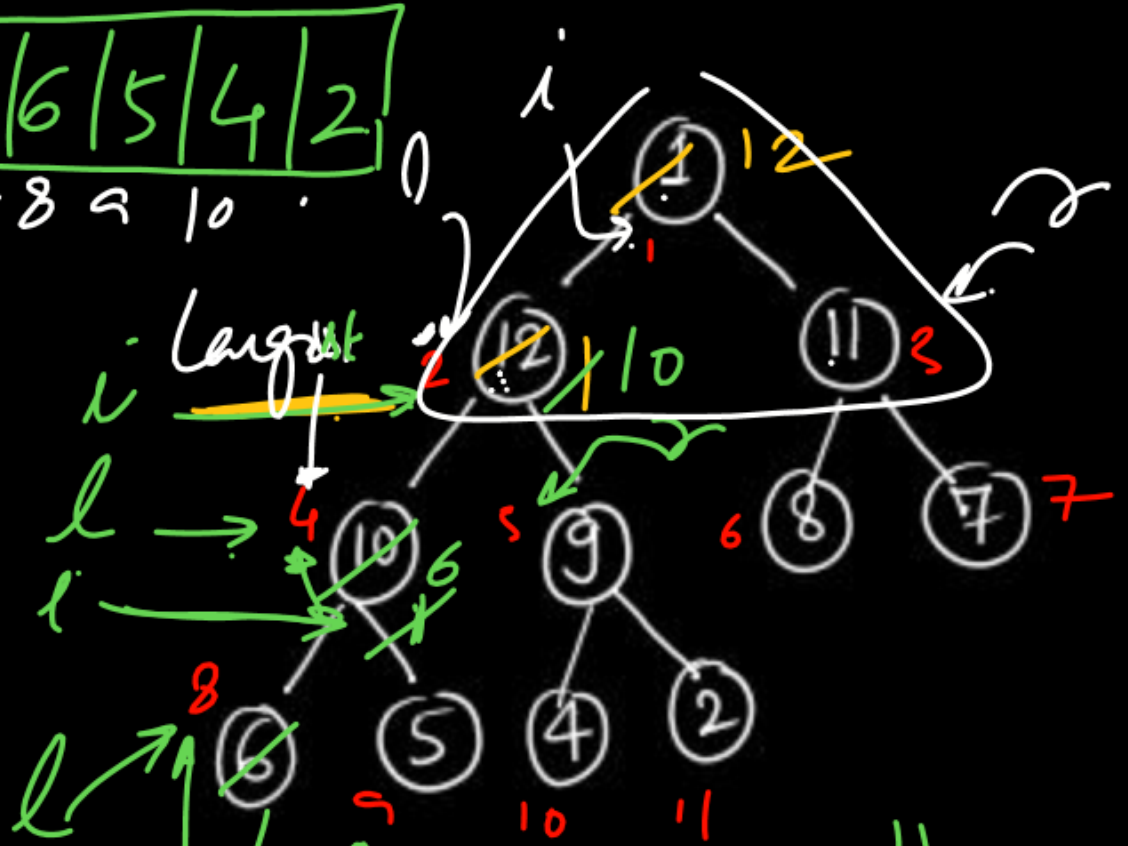
8 - if $\text{largest} \neq i$

9 - Then Exchange $A[i] \leftrightarrow A[\text{largest}]$

10 - Heapify(A, largest)

11 -

A	1	12	11	10	9	8	7	6	5	4	2
	1	2	3	4	5	6	7	8	9	10	



$O(\log n)$

Heapify(A, 8)

$l \leftarrow 16$

$r \leftarrow 17$

Running Time of Heapify Procedure:-

Recurrence Relation:

Worst Case:

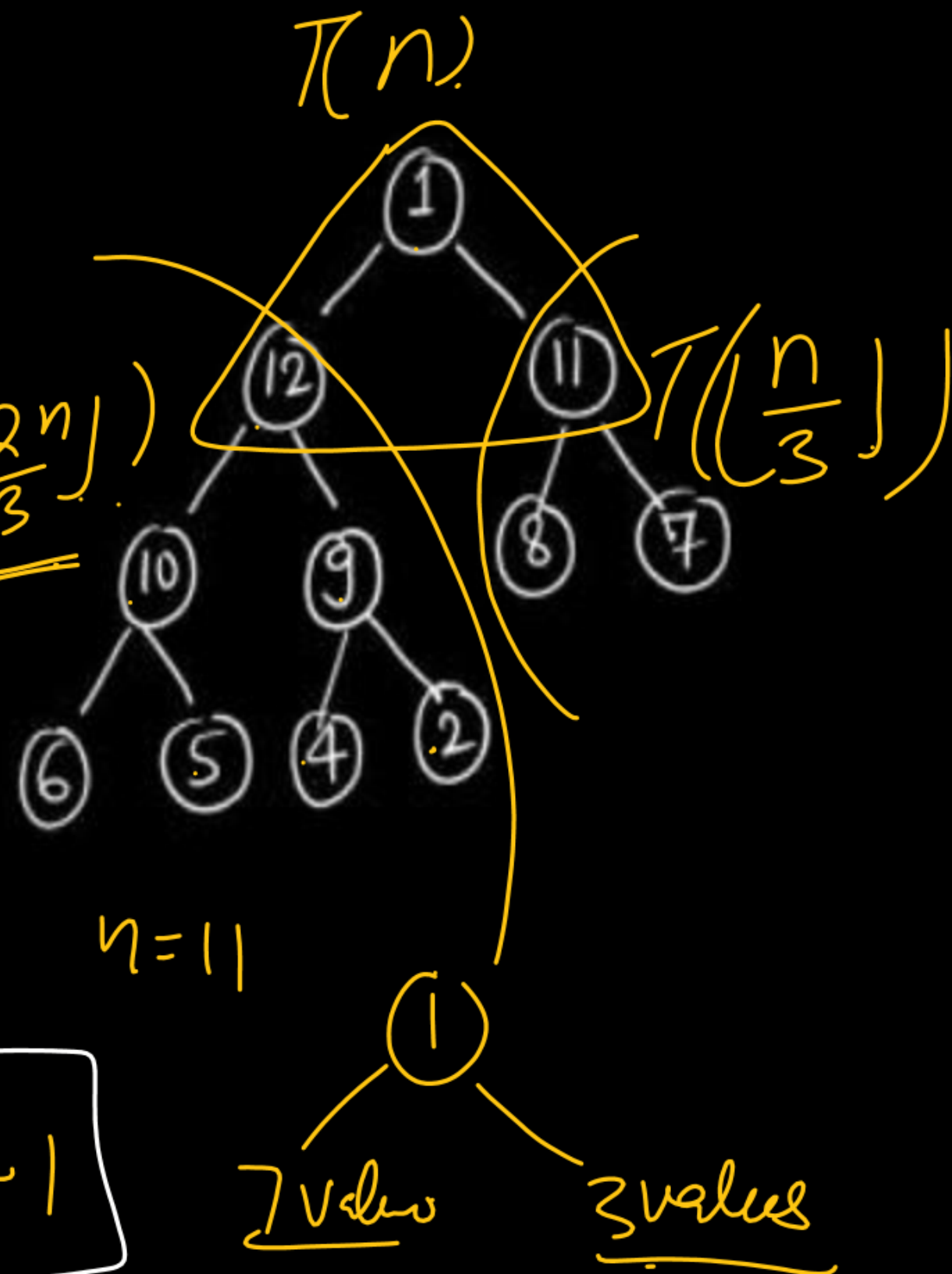
$$T(n) = \begin{cases} 1, & \text{if } n=1 \\ 1 + T(\lfloor \frac{2n}{3} \rfloor), & \text{if } n > 1 \end{cases}$$

$$T(n) \leq T(\frac{2n}{3}) + 1 \Rightarrow T(n) = T(\frac{2n}{3}) + 1$$

$$T(n) = O(\log_{3/2} n) \checkmark$$

$$T(n) = O(\log n)$$

$$T(\lfloor \frac{2n}{3} \rfloor)$$



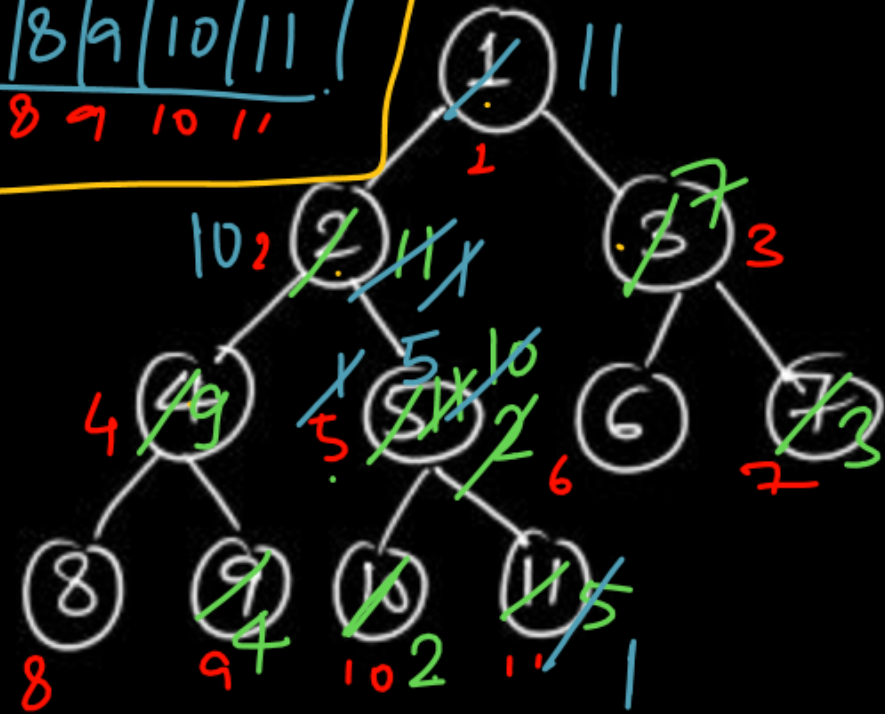
BuildHeap:- max heap

$$UB \Rightarrow O(n \log_2 n)$$

A	1	2	3	4	5	6	7	8	9	10	11
	1	2	3	4	5	6	7	8	9	10	11

$n=11$

nonleaf. leaf node.



Algo:- BuildHeap(A)

1 - Heapsize(A) $\leftarrow$ Length(A)

2 - for $i \leftarrow \lfloor \frac{\text{Length}(A)}{2} \rfloor$ down to 1

3 - $\hookrightarrow$ Heapify(A, i) // $\frac{n}{2} \times \log_2 n$

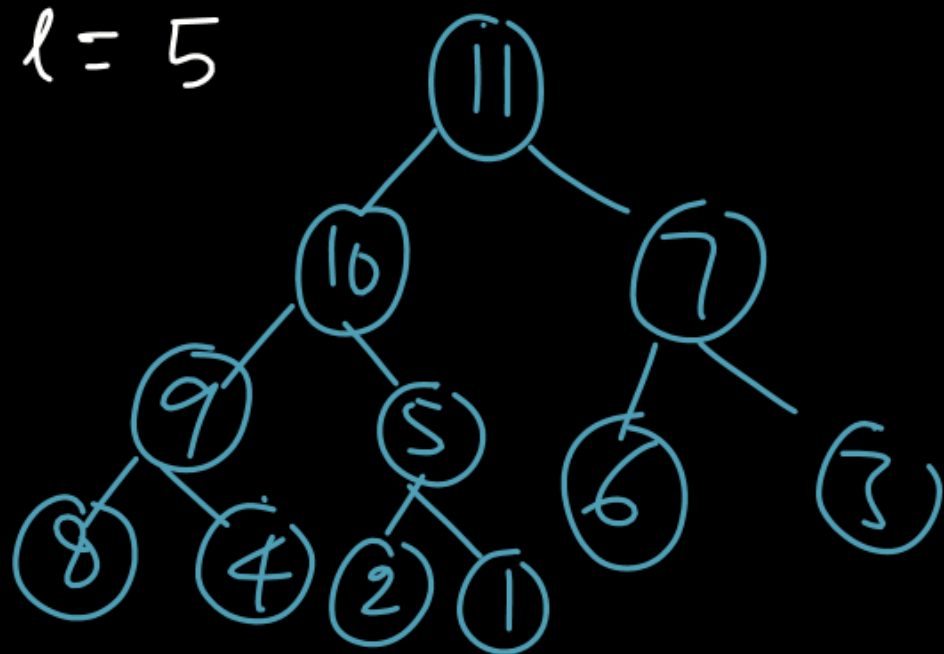
$$\text{leaf nodes} = A[\lfloor \frac{n}{2} \rfloor + 1 \dots n]$$

Call heapify = A[6 ... 11]

for

{ Nonleaf. }
{ Interval } $A[1 \dots \lfloor \frac{n}{2} \rfloor]$

$$= A[1 \dots 5]$$



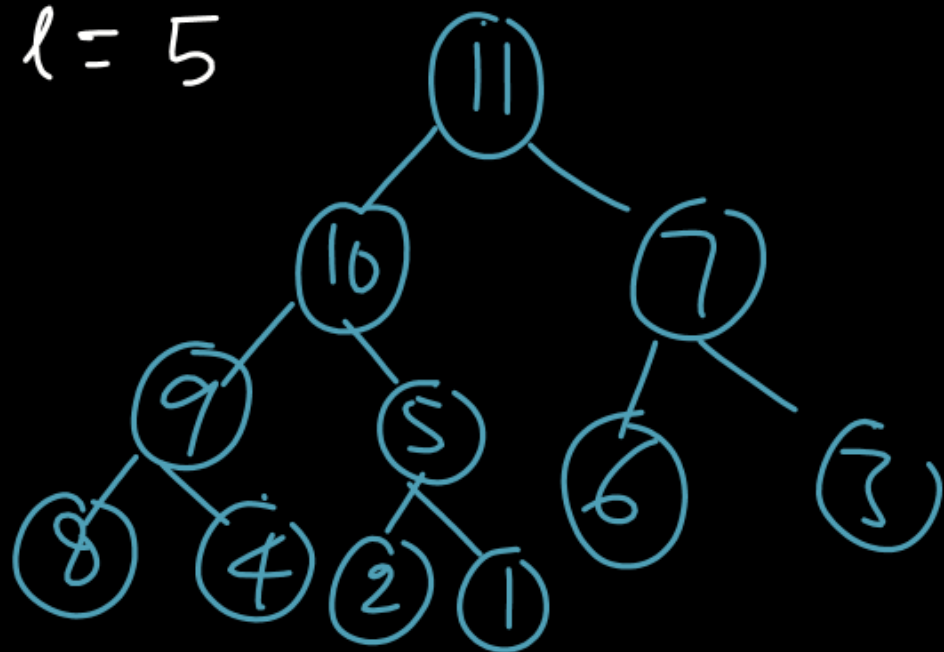
BuildHeap:- max heap

$UB \Rightarrow O(n \log_2 n)$

Algo:- BuildHeap(A)

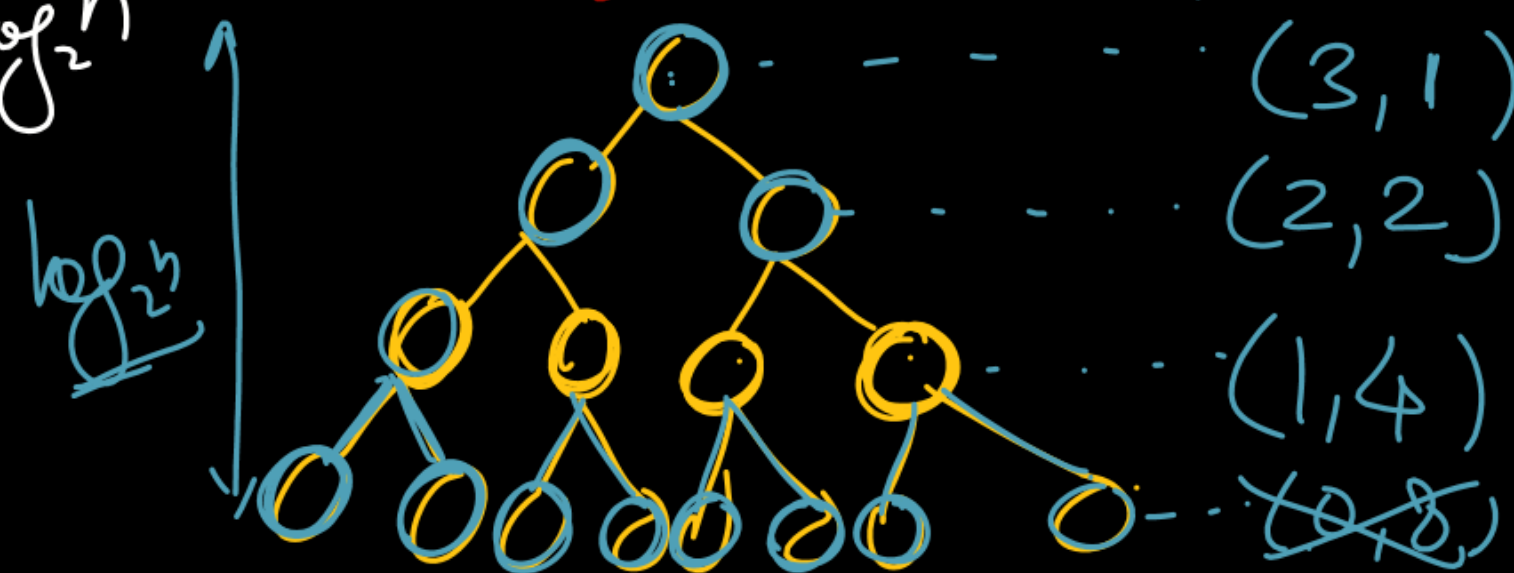
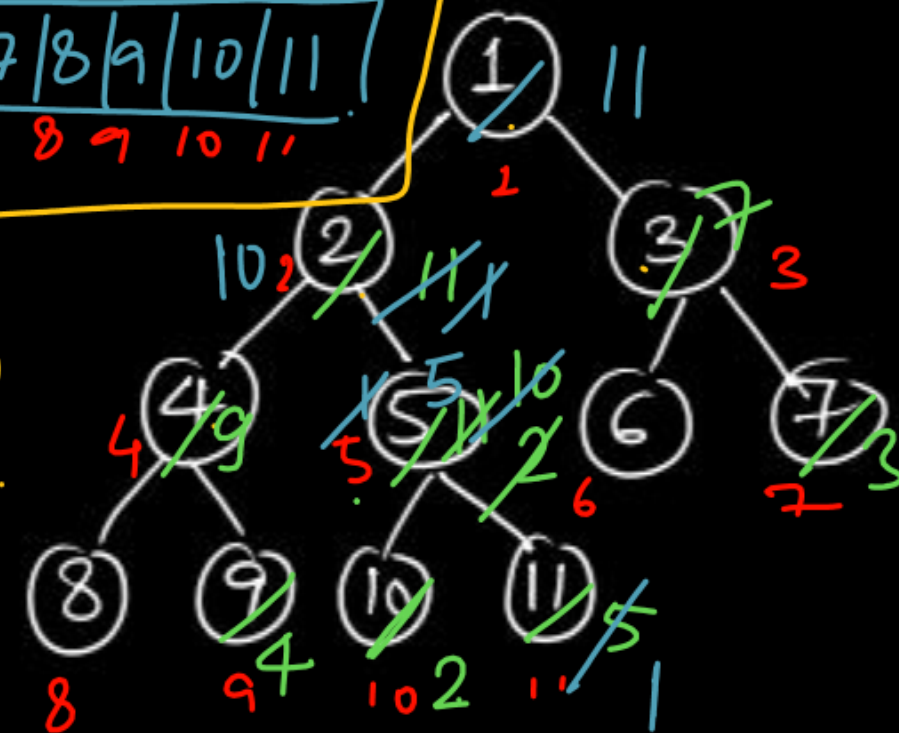
- 1 - $HeapSize(A) \leftarrow Length(A)$
- 2 - for $i \leftarrow \lfloor \frac{Length(A)}{2} \rfloor$ down to 1
- 3 - $\hookrightarrow Heapify(A, i) // \frac{n}{2} \times \log_2 n$

$l = 5$



A	1	2	3	4	5	6	7	8	9	10	11
	1	2	3	4	5	6	7	8	9	10	11

$n = 11$
 $\uparrow$ num leaf.
 $\uparrow$ leaf node.



BuildHeap:- max heap

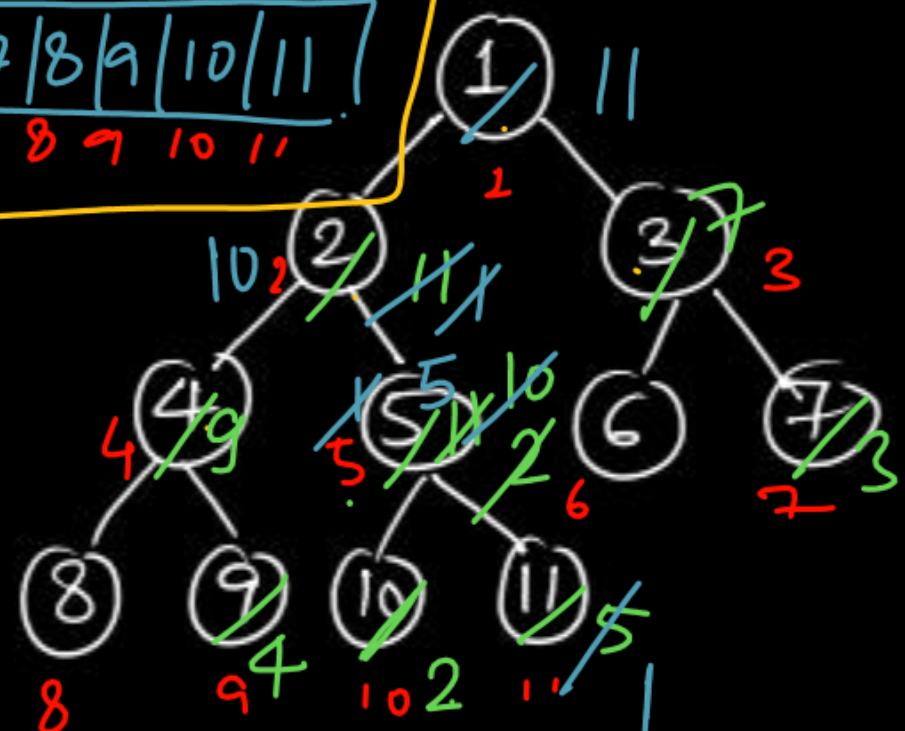
UB $\Rightarrow O(n \log n)$

Algo:- BuildHeap(A)

- 1 - Heapsize(A) $\leftarrow$ Length(A)
- 2 - for $i \leftarrow \lfloor \frac{\text{Length}(A)}{2} \rfloor$ down to 1
- 3 - $\hookrightarrow$ Heapify(A, i)

A	1	2	3	4	5	6	7	8	9	10	11
	1	2	3	4	5	6	7	8	9	10	11

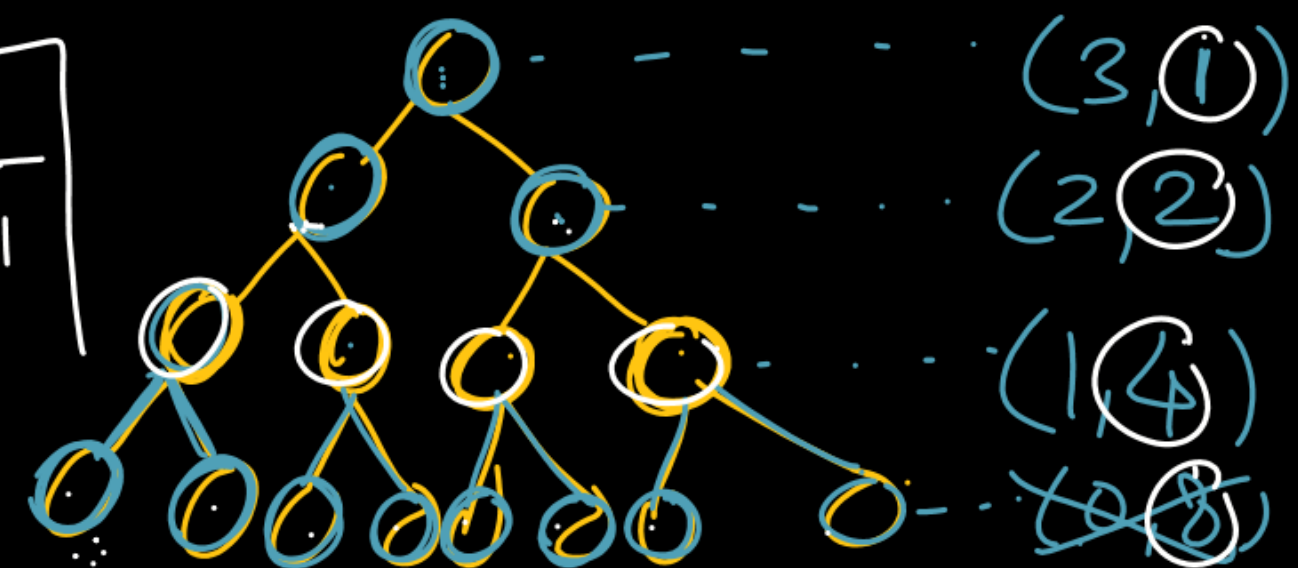
$n=11$
 $\uparrow$ num leaf.
 $\uparrow$ leaf node.



In heap no. of nodes on height(h) = $\left\lceil \frac{n}{2^{h+1}} \right\rceil$

$$\left\lceil \frac{15}{2^{2+1}} \right\rceil = \left\lceil \frac{15}{8} \right\rceil = 2$$

$$\left\lceil \frac{15}{2^{3+1}} \right\rceil = 1$$



$$\text{Total} = 1 \times 4 + 2 \times 2 + 3 \times 1 = 11$$

BuildHeap:- max heap

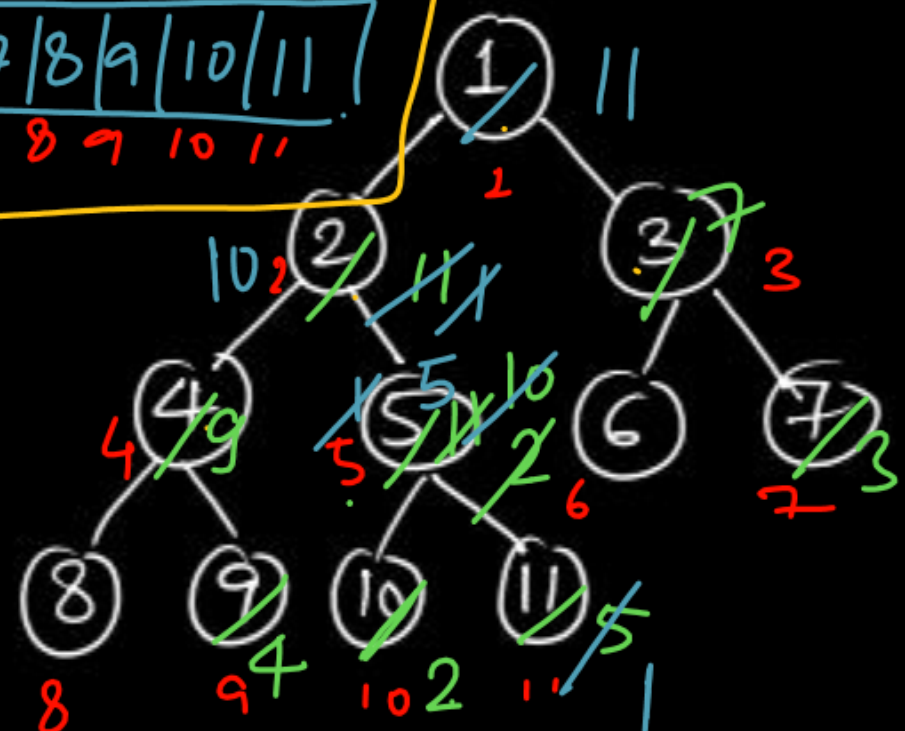
UB $\Rightarrow O(n \log n)$

Algo:- BuildHeap(A)

- 1 - Heapsize(A) $\leftarrow$ Length(A)
- 2 - for $i \leftarrow \lfloor \frac{\text{Length}(A)}{2} \rfloor$ down to 1
- 3 - $\hookrightarrow$ Heapify(A, i)

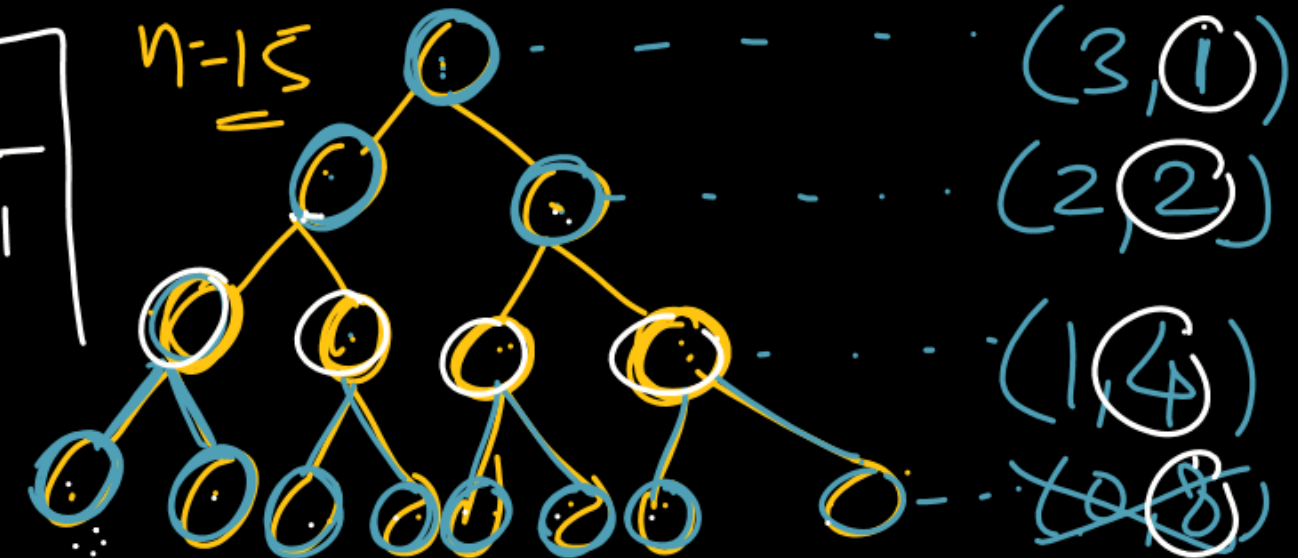
A	1	2	3	4	5	6	7	8	9	10	11
	1	2	3	4	5	6	7	8	9	10	11

$n=11$
 $\uparrow$
 non leaf. leaf node.



In heap no. of nodes on height(h) = $\left\lceil \frac{n}{2^{h+1}} \right\rceil$
 Total cost To Build max/min heap

$$\sum_{h=0}^{\log n} h \times \left\lceil \frac{n}{2^{h+1}} \right\rceil$$



Total = $1 \times 5 + 2 \times 2 + 3 \times 1$
 $= 11$

BuildHeap:- max heap

UB $\Rightarrow O(n \log n)$

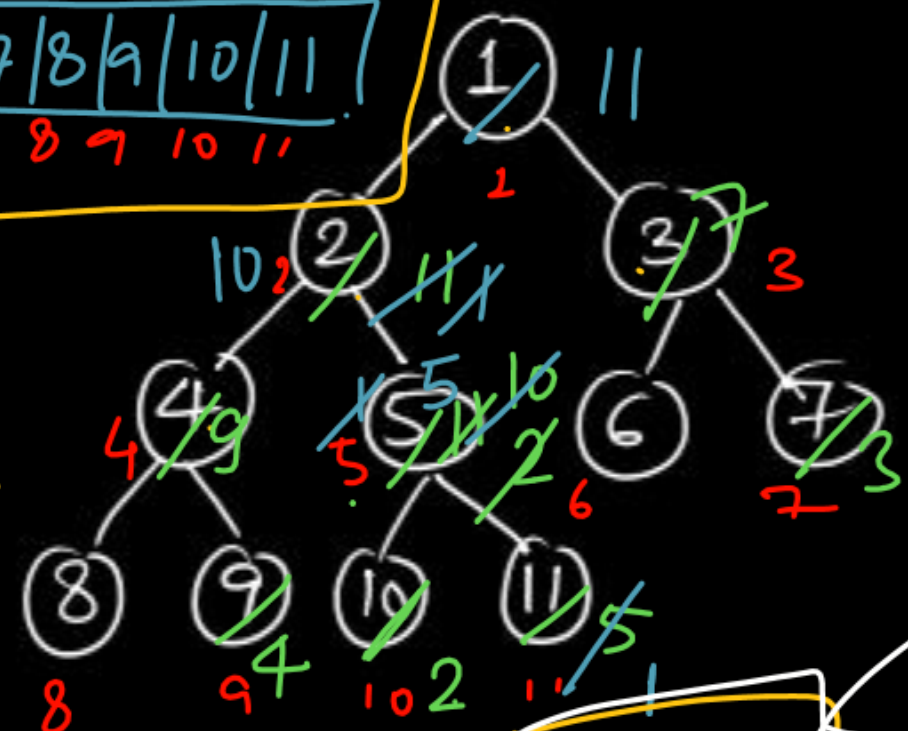
Algo:- BuildHeap(A)

- 1 - Heapsize(A) $\leftarrow$ Length(A)
- 2 - for $i \leftarrow \lfloor \frac{\text{Length}(A)}{2} \rfloor$ down to 1
- 3 - $\hookrightarrow$ Heapify(A, i)

TC = $O(n)$

A	1	2	3	4	5	6	7	8	9	10	11
	1	2	3	4	5	6	7	8	9	10	11

$n=11$
 $\uparrow$ num leaf.
 $\uparrow$ leaf node.



$\Rightarrow \frac{n}{2} \times C \Rightarrow O(n)$
 $\frac{n}{2} [1 + \frac{3}{2} + \frac{4}{16} + \dots]$

In heap no. of nodes on height (h) = $\lceil \frac{n}{2^{h+1}} \rceil$
 Total cost To Build max/min heap

$$\sum_{h=0}^{\log n} h \times \lceil \frac{n}{2^{h+1}} \rceil \Rightarrow \frac{n}{2} \sum_{h=0}^{\log_2 n} \left(\frac{h}{2^h} \right) \Rightarrow \frac{n}{2} \left[0 + \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + \dots \right]$$

Build Heap Tighter Analysis:-

$$TC = O(n)$$

$$\text{No of Nodes on height } h \Rightarrow \left\lceil \frac{n}{2^{h+1}} \right\rceil$$

Operations on Heap:-

i) HeapIncreaseKey:-

A

100	50	20	2	3	10	5
1	2	3	4	5	6	7

Algo: HeapIncrease(A, i, key)

1> if (key < A[i])

2> "Error"

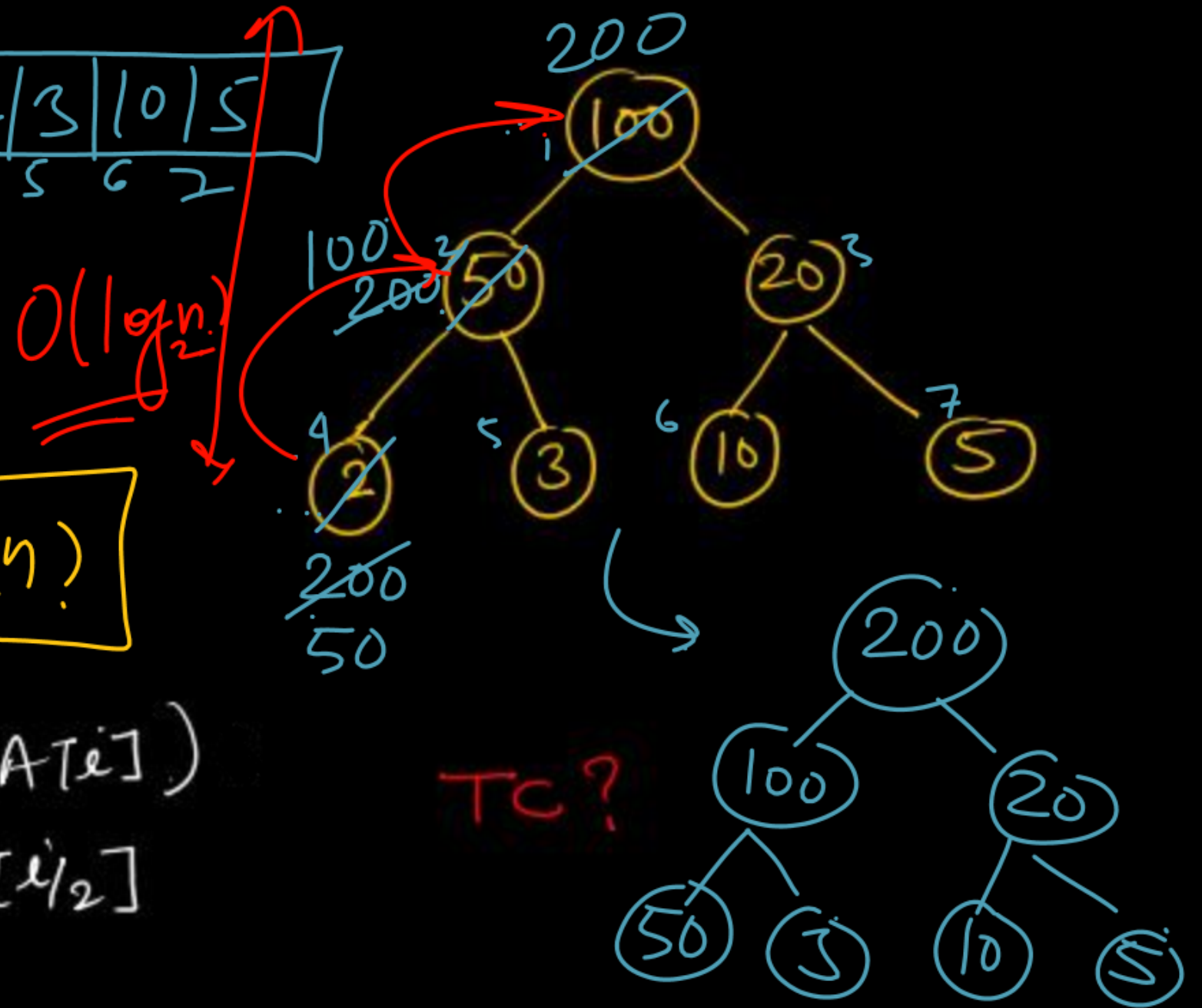
3> A[i] = key

4> While (i > 1 and A[i/2] < A[i])

5> Exchange A[i] ↔ A[i/2]

6> i = i/2

$$TC = O(\log_2 n)$$



ii) HeapDecreaseKey:- Similar to HeapIncreaseKey

TC?

Operations on Heap:-

i) HeapIncreaseKey:-

Algo: HeapIncrease(A, i, key)

1> if ($key < A[i]$)

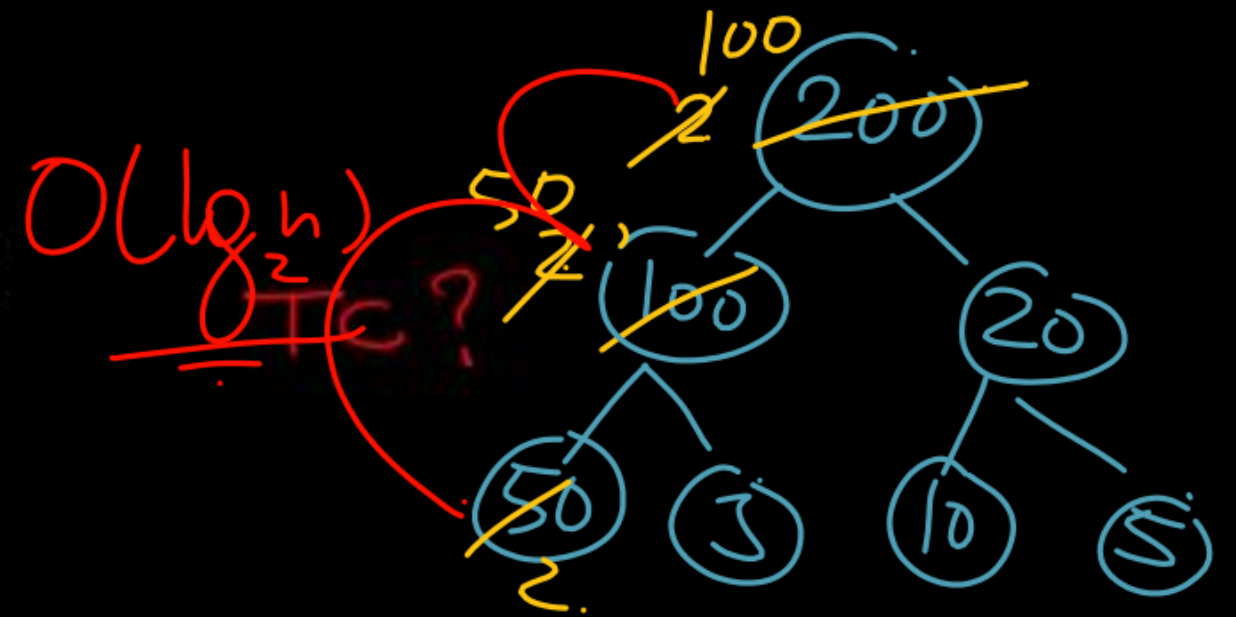
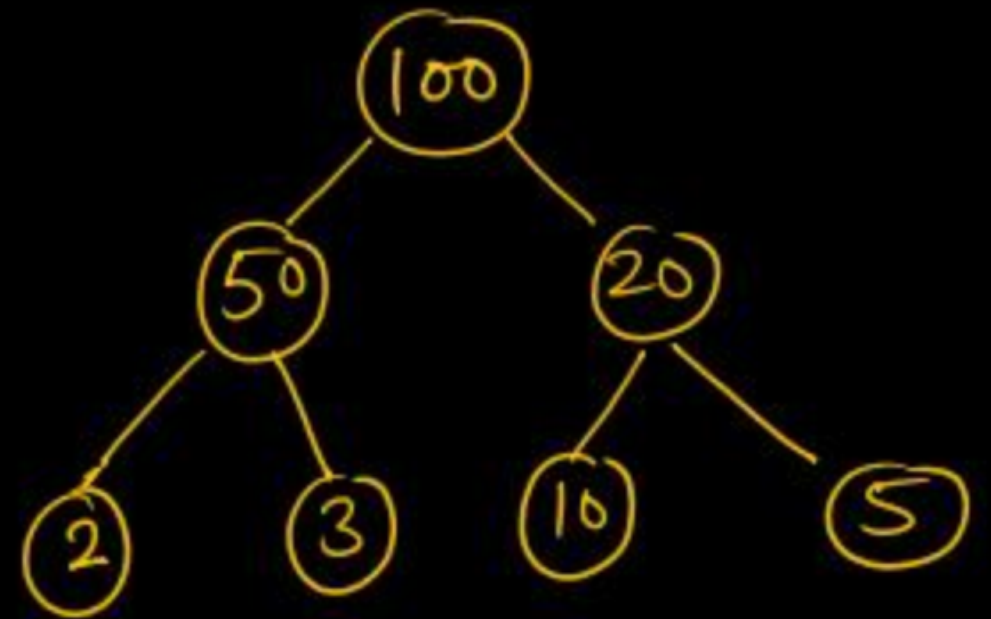
2> "Error" $TC = O(\log_2 n)$

3> $A[i] = key$

4> While ($i > 1$ and $A[i/2] < A[i]$)

5> Exchange $A[i] \leftrightarrow A[i/2]$

6> $i = i/2$



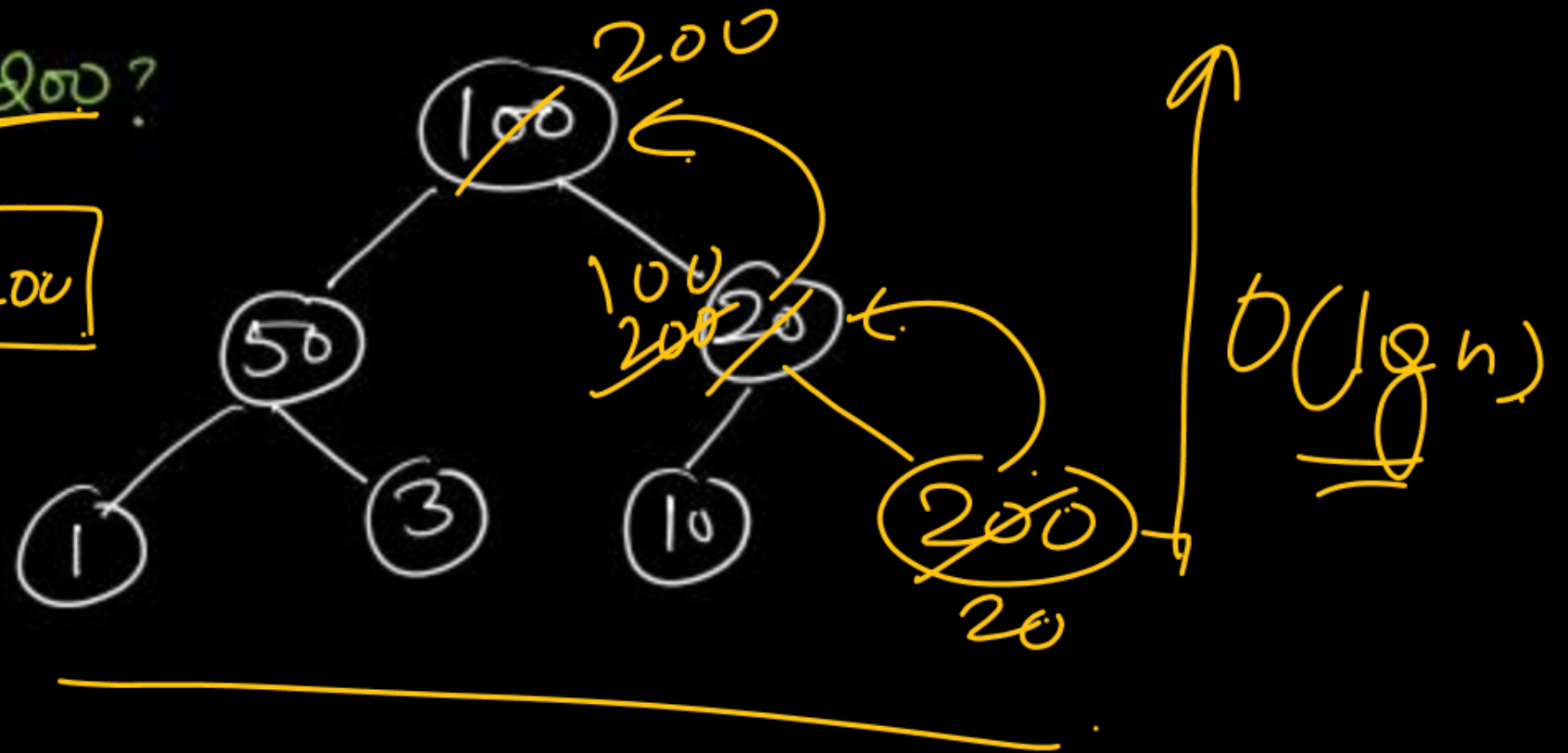
ii) HeapDecreaseKey:- Similar to HeapIncreaseKey TC?
 $\rightarrow O(\log_2 n)$

iii) Insert into max heap:- Insert 200?

TC? A

100	50	20	1	3	10	200
-----	----	----	---	---	----	-----

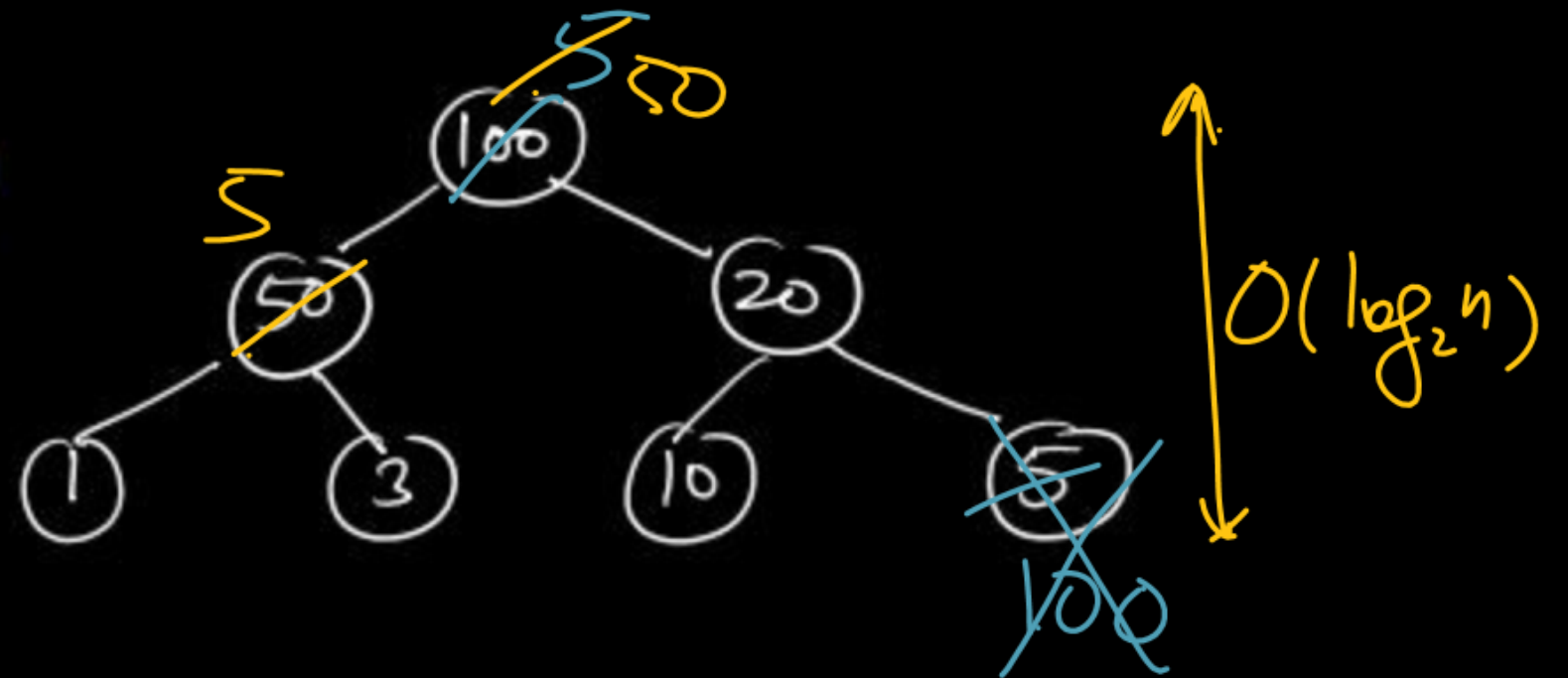
$O(\log_2 n)$



iv) Extract max: Extract 100?

TC?

100	50	20	1	3	10	5
-----	----	----	---	---	----	---



Extract min = $O(\lg n)$

Min heap:-

Time Complexity

1) Search min element? $O(1)$

2) Search max element? $O(n)$

3) Find n th min element? $O(n)$

4) Search $(n-1)$ th min element? $O(n)$

5) Search $(\frac{n}{2}+1)$ th min element? $O(n)$ 2nd min

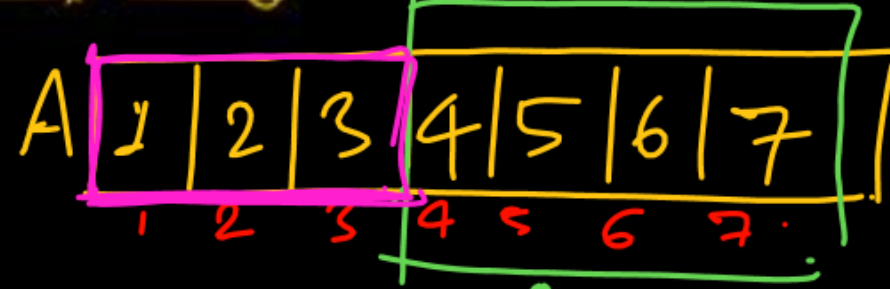
6) Search 7th min element? $2^7 - 1 = O(127)$ $O(1)$

7) Delete min element? $O(\lg n)$

8) Delete max element = ?

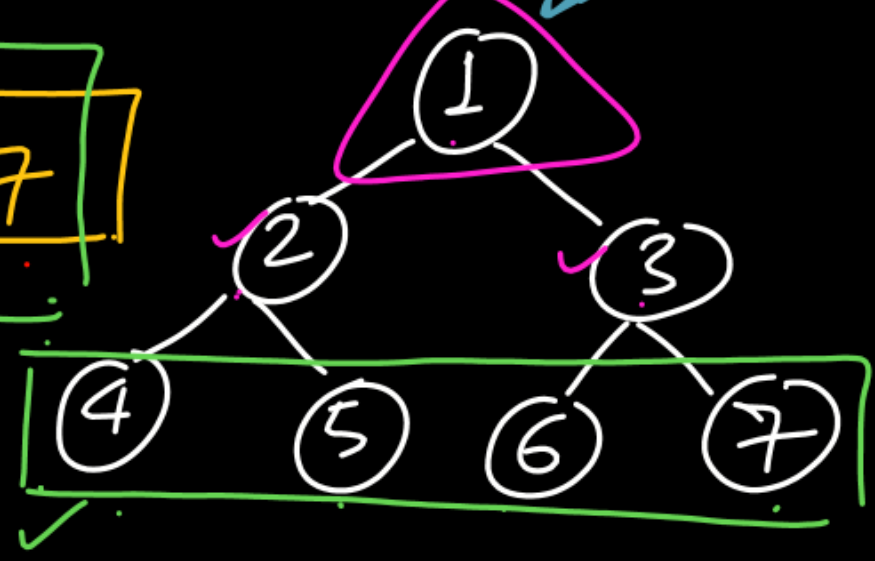
9) Delete n th min?

10) Delete 7th min?

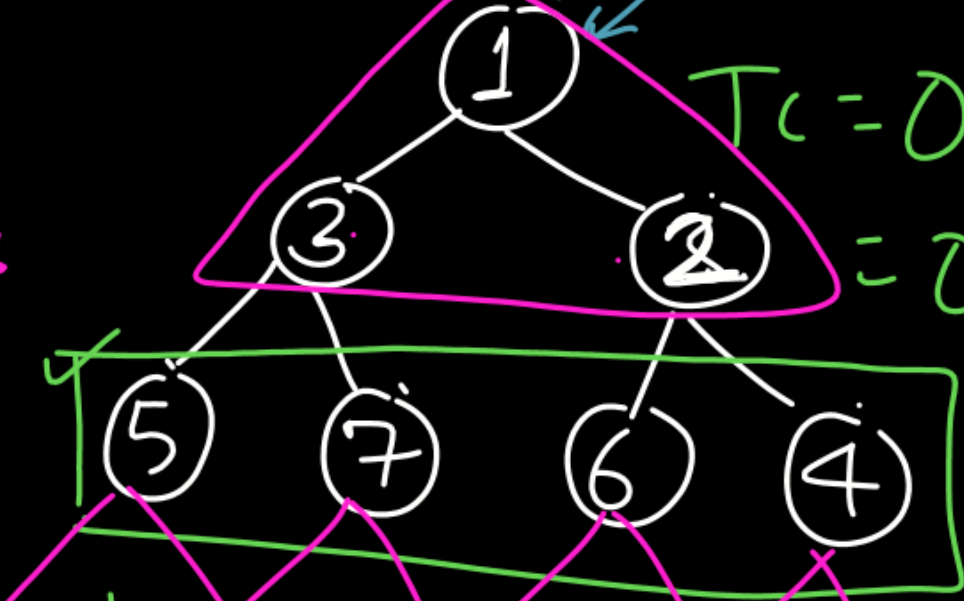


leaf node

min heap



min heap



$T_c = O(\frac{n}{2})$

$= O(n)$

$2^0 + 2^1 = 2^2 - 1$

3rd min =

$2^0 + 2^1 + 2^2 = 2^3 - 1 = 7$

leaf node = $A \left[\left\lfloor \frac{n}{2} \right\rfloor + 1 \dots n \right]$

Min heap:-

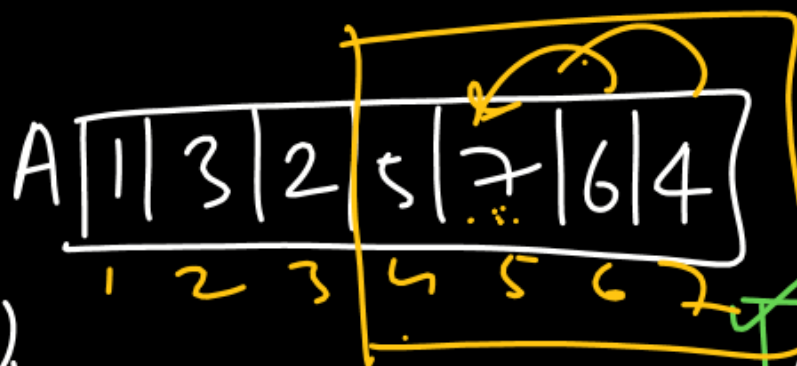
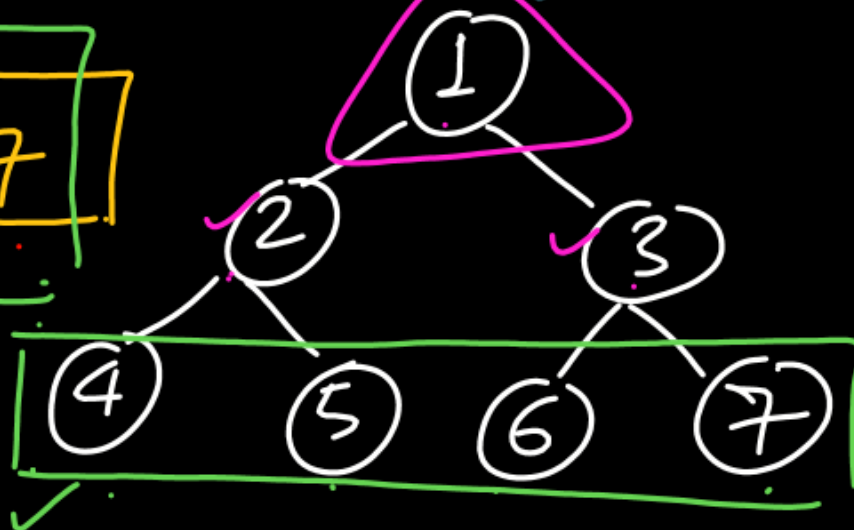
Time Complexity

- 1) Search min element? $O(1)$
- 2) Search max element? $O(n)$
- 3) Find n^{th} min element? $O(n)$
- 4) Search $(n-1)^{\text{th}}$ min element? $O(n)$
- 5) Search $(\frac{n}{2}+1)^{\text{th}}$ min element? $O(n)$
- 6) Search 7^{th} min element? $O(1)$
- 7) Delete min element? $O(\log n)$
- 8) Delete max element? $O(n)$
- 9) Delete n^{th} min? $O(n)$
- 10) Delete 7^{th} min? $O(\log n)$



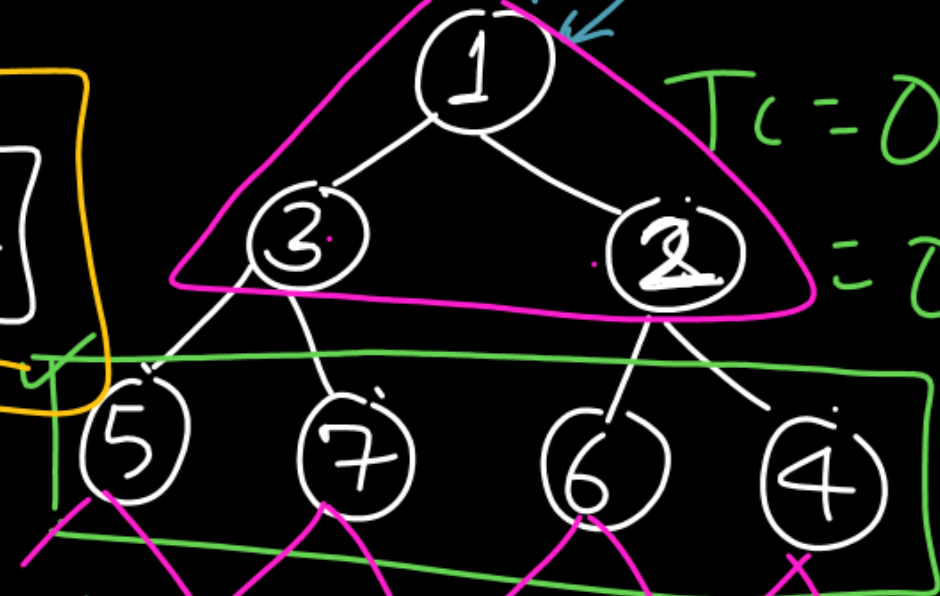
leaf node

min heap



Search = $O(4)$
 $= O(n)$
 $O(n)$

min heap



leaf node = $A \left[\left\lfloor \frac{n}{2} \right\rfloor + 1 \dots n \right]$

$T_c = O(\frac{n}{2})$
 $= O(n)$

Heap with Binary Search:-

Binary search:

No. of Comparisons $\Rightarrow \log \log n$

No. of Swaps $\Rightarrow \log n$

n Binary search = $O(\log n)$

Linear search = $O(n)$

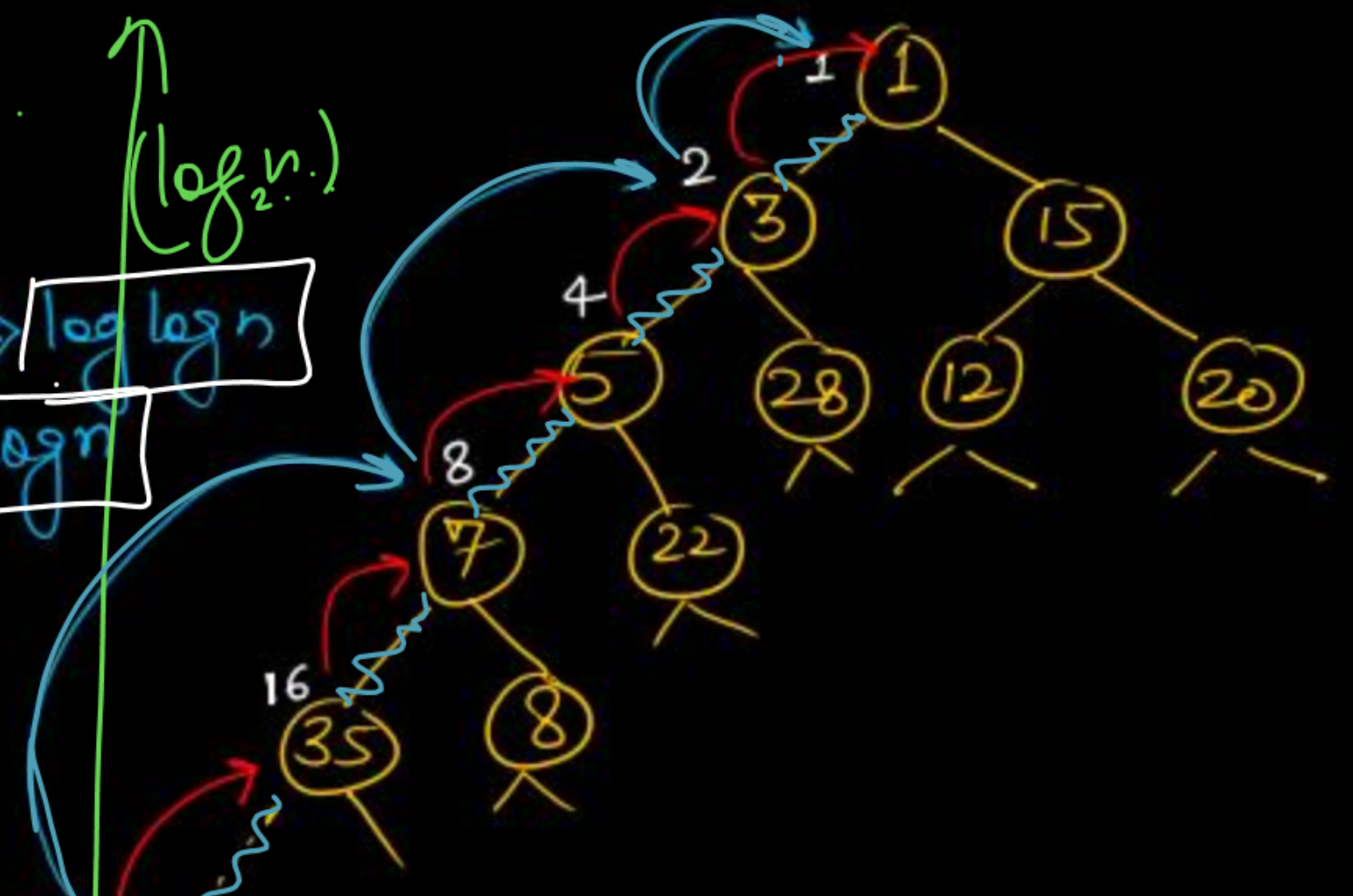
$\log n$ Binary search = $O(\log \log n)$

Linear = $O(\log n)$

Linear search

(K_1) new element

No. of Comparisons $\Rightarrow \log n$
No. of Swaps $\Rightarrow \log n$



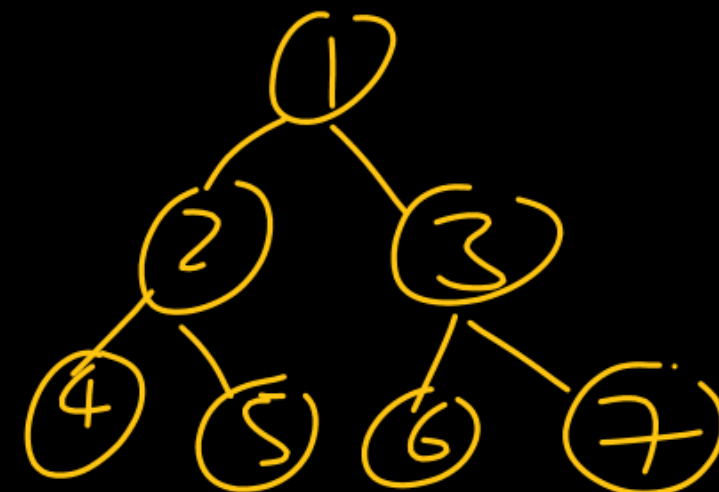
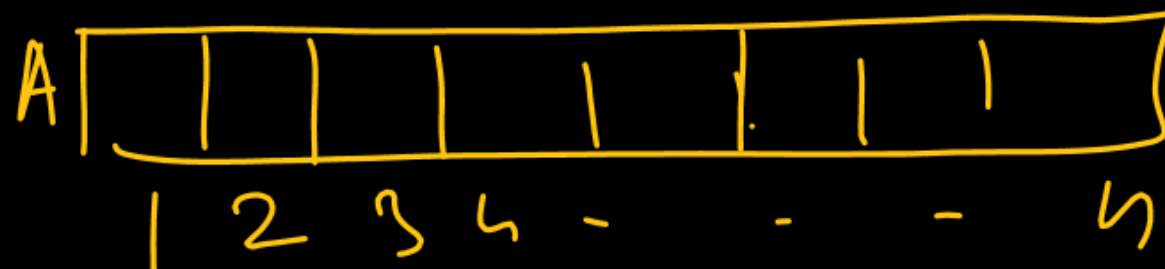
Q) Consider a binary min-heap containing 105 distinct elements. Let k be the index (in the underlying array) of the maximum element stored in the heap. The number of possible values of k is

A) 53

B) 52

C) 27

D) 1



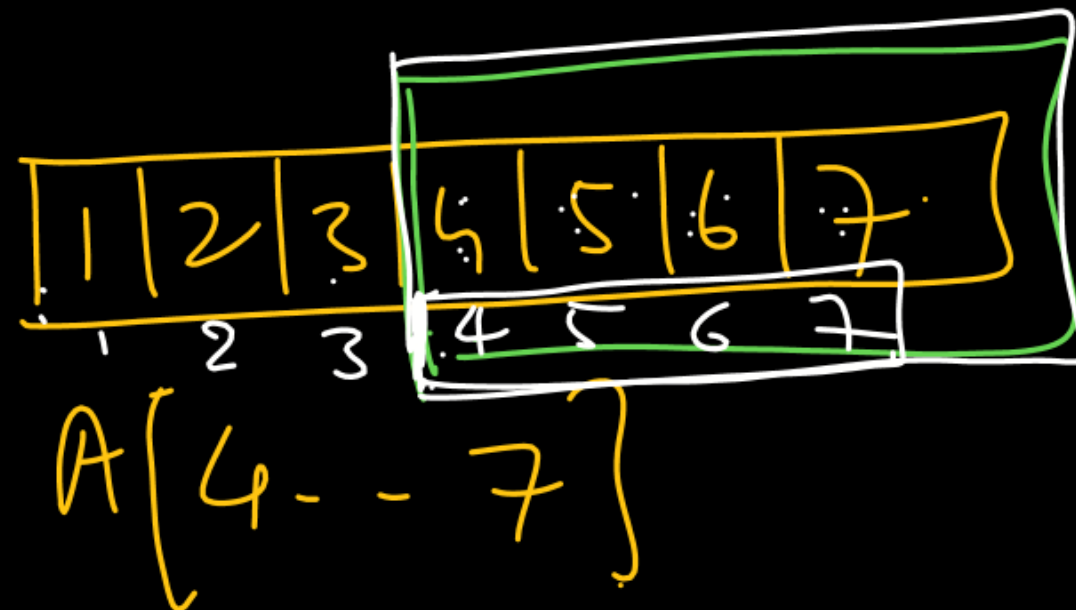
$$\left\lfloor \frac{n}{2} \right\rfloor + 1$$

$$\Rightarrow \left\lfloor \frac{105}{2} \right\rfloor + 1$$

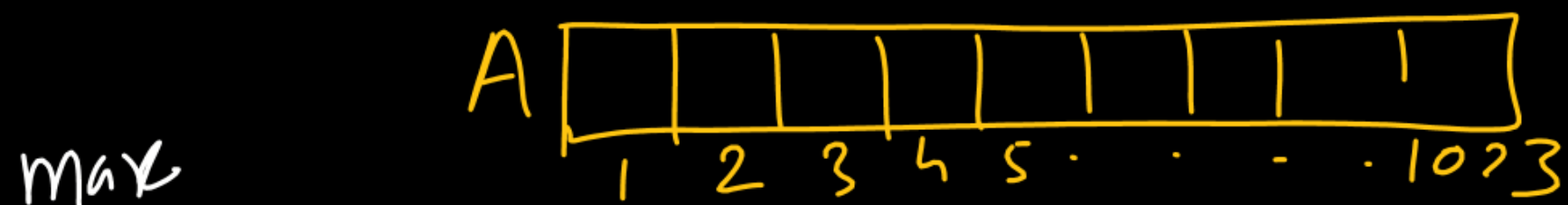
$$52 + 1 = 53$$

$$A \left[\left\lfloor \frac{n}{2} \right\rfloor + 1 - \dots n \right]$$

$$n=7$$



Consider the array representation of a binary min-heap containing 1023 elements. The minimum number of comparisons required to find the maximum in the heap is _____.



max $\Rightarrow \underline{(n-1)}$

Compar

$= A[\lfloor \frac{1023}{2} \rfloor + 1 - \dots - 1023]$

Q) Which one of the following sequences when stored in an array at locations $A[1], \dots, A[10]$ forms a max-heap?

A) 23, 17, 10, 6, 13, 14, 1, 5, 7, 12

B) 23, 17, 14, 7, 13, 10, 1, 5, 6, 12

C) 23, 17, 14, 6, 13, 10, 1, 5, 7, 15

D) 23, 14, 17, 1, 10, 13, 16, 12, 7, 5

Q) Let H be a binary min-heap consisting of n elements implemented as an array. What is the worst case time complexity of an optimal algorithm to find the maximum element in H?

A) $\Theta(1)$

B) $\Theta(\log n)$

☒ C) $\Theta(n)$

D) $\Theta(n \log n)$

Q) In a heap with n elements with the smallest element at the root, the 7th smallest element can be found in time

A	$\Theta(n \log n)$
B	$\Theta(n)$
C	$\Theta(\log n)$
D	$\Theta(1)$

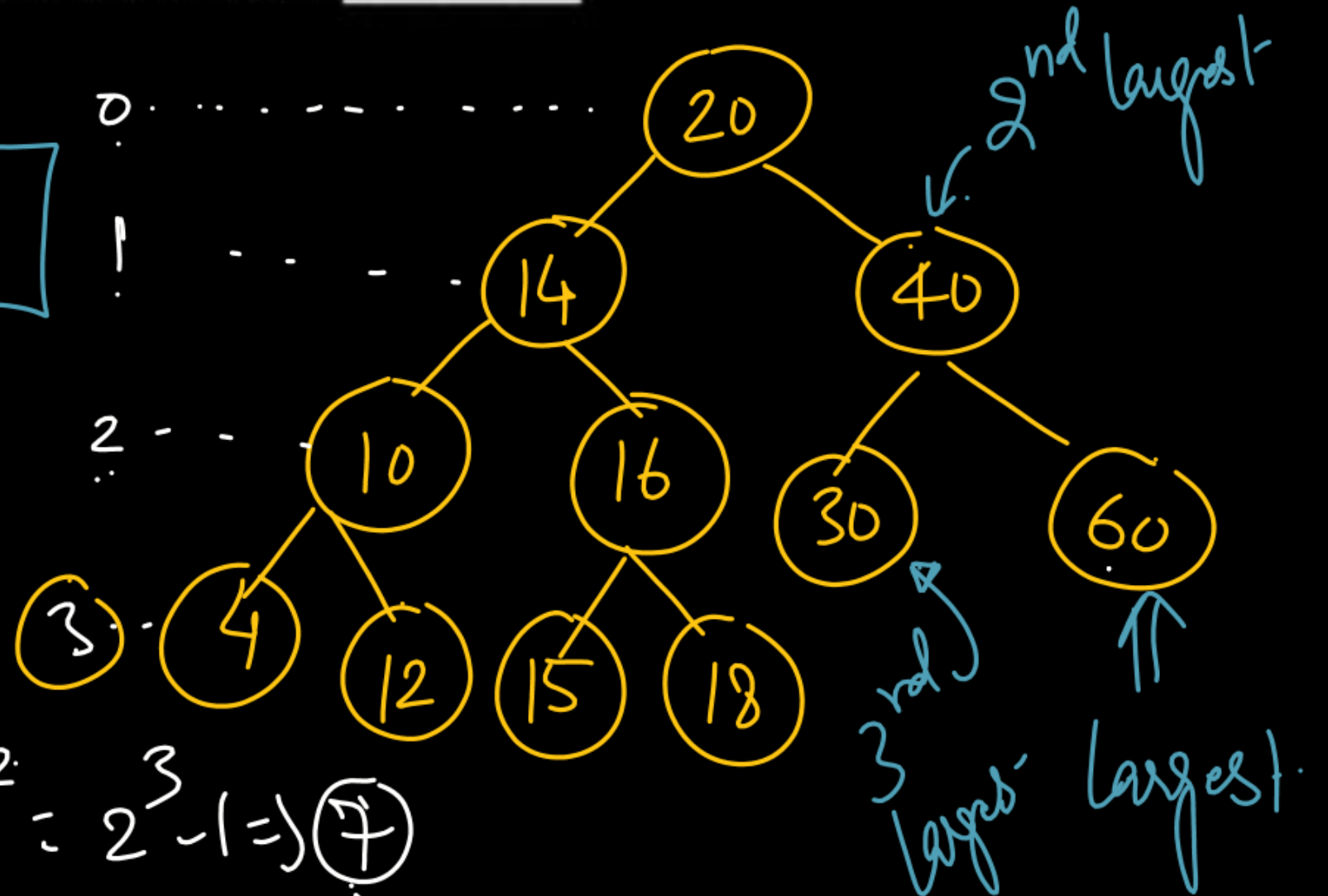
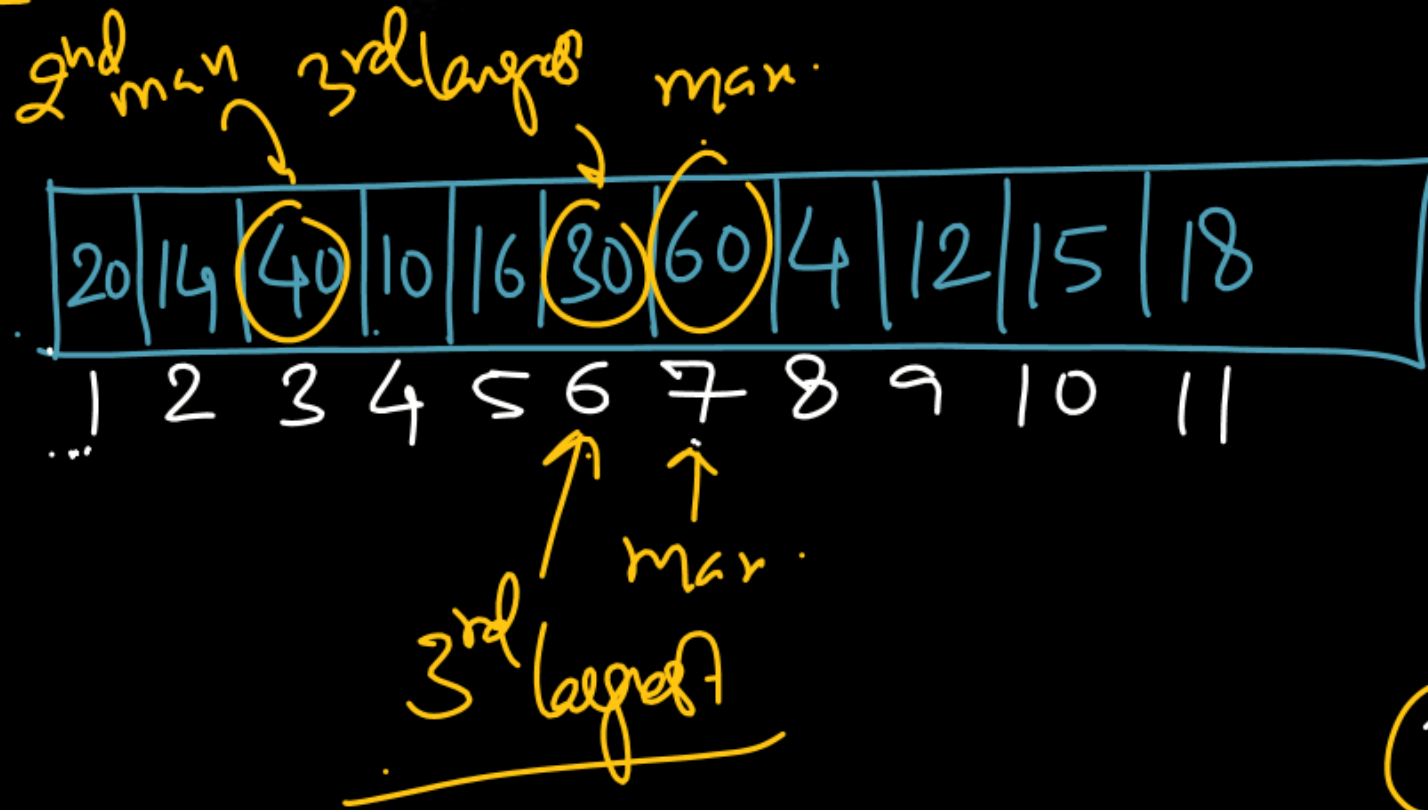
min heap:

$$2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^5 + 2^6 = 2^7 - 1$$

$$= \underline{127}$$

$$\underline{\Theta(1)}$$

Q) Suppose a binary search tree with 1000 distinct elements is also a complete binary tree. The tree is stored using the array representation of binary heap trees. Assuming that the array indices start with 0, the 3rd largest element of the tree is stored at index _____.



$$2^0 + 2^1 + 2^2 = 2^3 - 1 = 7$$

$$3^{\text{rd}} \text{ largest} = 7 - 1 = 6$$

Q) Suppose a binary search tree with 1000 distinct elements is also a complete binary tree. The tree is stored using the array representation of binary heap trees. Assuming that the array indices start with 0, the 3rd largest element of the tree is stored at index _____.

$$n = 1000$$

$$h = \lfloor \log_2 n \rfloor \Rightarrow \lfloor \log_2 1000 \rfloor = 9$$

$$2^0 + 2^1 + 2^2 + \dots + 2^8 = 2^9 - 1$$

$$\text{Index of largest} = 511$$

$$\text{3rd largest} \Rightarrow 510$$

$$\therefore \text{index starts from } 0$$

$$\Rightarrow 510 - 1 = 509$$

509

Ans





Shailendra Singh GATE CSE/DA

THANKYOU



GeeksforGeeks GATE